

Kobuki Native Windows Project

Visual Studio C++ project walkthrough and usage examples

Native Windows C++ application
Controls Kobuki through a COM port
Implements a small teaching-friendly API
Includes protocol tests and classroom examples



Kobuki Native Windows Project

Project walkthrough, file structure, code explanation, and usage examples for the odometry + gyro version

What this project is

A native Windows C++ application for controlling a Kobuki base through a COM port. It reads core sensors, gyro data, and encoder-based odometry, and it runs directly in Visual Studio 2026.

```
kobuki::Parameters p;  
p.device_port = "COM4";  
  
kobuki::Kobuki robot;  
robot.init(p);  
robot.enable();  
robot.setBaseControl(0.15, 0.0);
```

Main capabilities

- Base motion control
- COM port communication
- Core sensor reading
- Gyro / inertial data
- Encoder odometry
- Console demo application

Build and Run Workflow

From Visual Studio to a moving Kobuki

1 Connect Kobuki by USB and power it on

2 Find the COM port in Device Manager

3 Open KobukiNativeVS.sln

4 Select Debug | x64 and build

5 Run with Ctrl + F5 and enter COM4

KobukiNativeVS.exe COM4

Detected COM ports:

COM3

COM4

Enter Kobuki COM port: COM4

Keyboard controls

W forward

S backward

A rotate left

D rotate right

X or Space stop

R sensors

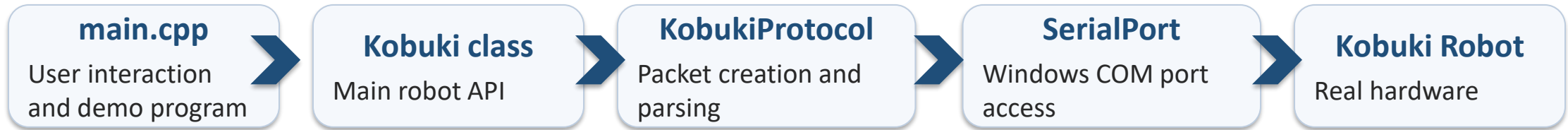
G gyro

O odometry

Z reset odometry

Q quit

High-Level Architecture



The application sends velocity commands to the robot.

The robot sends feedback packets back through the serial link.

The project parses these packets into C++ data structures.

Odometry is updated from wheel encoders, and heading is read from the gyro.

SerialPort.h and SerialPort.cpp

This module wraps the Windows COM port API.

It opens the selected port and configures 115200 baud, 8 data bits, 1 stop bit, and no parity.

It provides simple read and write functions for raw bytes.

It also provides a helper that lists available COM ports.

```
SerialPort serial;  
serial.open("COM4");  
  
serial.write(packet.data(),  
packet.size());  
size_t n = serial.read(buffer,  
bufferSize);
```

Kobuki Packet Layout

How a command packet is organized



AA 55 06 01 04 96 00 00 00 95

Example: forward at 0.15 m/s

Speed = 150 mm/s = 0x0096

Radius = 0 mm

Checksum = XOR of payload bytes

The header marks the start of a packet.
The payload contains command-specific bytes.
The checksum helps detect corrupted data.

Protocol Format Used in the Project

Serial link: 115200 baud, 8N1

Packet header: AA 55

Checksum: XOR

Base Control command ID: 0x01

Core Sensor feedback ID: 0x01

Inertial Sensor feedback ID: 0x04

AA 55 | length | payload... |
checksum

payload for motion command
= sub-id + data length + speed +
radius

Kobuki.h – Public API

Kobuki.h defines the user-facing robot API.

It intentionally looks simple and clean for teaching.

The class hides serial communication, protocol parsing, and thread management.

```
void init(Parameters& parameters);  
bool enable();  
bool disable();  
void setBaseControl(const double& linearVelocity,  
                    const double& angularVelocity);  
  
CoreSensors::Data getCoreSensorData() const;  
Inertia::Data getInertiaData() const;  
OdometryData getOdometryData() const;  
void resetOdometry();
```

setBaseControl(linear, angular)

The main function used to move the robot

```
robot.setBaseControl(0.15, 0.0); // forward  
robot.setBaseControl(-0.15, 0.0); // backward  
robot.setBaseControl(0.0, 0.8); // rotate left  
robot.setBaseControl(0.0, -0.8); // rotate right  
robot.setBaseControl(0.0, 0.0); // stop
```

Parameters

linearVelocity: meters per second

angularVelocity: radians per second

The protocol layer converts these to speed and radius values.

linear > 0



moves forward

angular ≠ 0



turns robot

Core Data Structures

CoreSensors::Data

Stores time stamp, bumpers, wheel drop, cliff, left encoder, right encoder, PWM values, buttons, charger, battery, and overcurrent.

Inertia::Data

Stores gyro angle, angle rate, and acceleration bytes. It also provides helper functions for degrees and radians.

OdometryData

Stores x, y, heading, linear velocity, angular velocity, left distance, and right distance.

```
double batteryVoltage() const;
bool bumperLeft() const;
bool bumperCenter() const;
bool bumperRight() const;
double angleRadians() const;
double angularVelocityRadiansPerSecond() const;
```

CoreSensors::Data

Readable C++ representation of Kobuki sensor values

```
struct Data {  
    uint16_t time_stamp;  
    uint8_t bumper;  
    uint8_t wheel_drop;  
    uint8_t cliff;  
    uint16_t left_encoder;  
    uint16_t right_encoder;  
    int8_t left_pwm;  
    int8_t right_pwm;  
    uint8_t buttons;  
    uint8_t charger;  
    uint8_t battery;  
    uint8_t overcurrent;  
};
```

```
auto data = robot.getCoreSensorData();  
  
std::cout << data.left_encoder;  
std::cout << data.right_encoder;  
std::cout << data.batteryVoltage();  
  
if (data.bumperCenter()) {  
    robot.setBaseControl(0.0, 0.0);  
}
```

Helper methods make bit fields easier to teach:
bumperLeft(), bumperCenter(), cliffRight(),
batteryVoltage().

PWM (Pulse Width Modulation)

Odometry Calculation

Odometry is computed from the left and right wheel encoders.

Each encoder tick is converted into metres using TicksToMeters.

The average of the two wheel distances gives the forward motion.

The difference between the wheel distances gives the heading change.

```
leftDistance  = deltaLeftEncoder  *  
TicksToMeters;  
rightDistance = deltaRightEncoder *  
TicksToMeters;  
  
distance = (leftDistance + rightDistance) /  
2.0;  
deltaHeading = (rightDistance - leftDistance)  
/ WheelBaseMeters;
```

What left_distance and right_distance mean

They are the total travelled distances of the left and right wheels since the last reset.

Gyro / Inertial Sensor

The project reads the Kobuki inertial packet and stores angle and angle_rate.

The raw values are reported in hundredths of a degree.

Helper functions convert them into degrees, radians, degrees/s, and rad/s.

getHeading() returns the gyro heading relative to the last init or reset.

```
auto g = robot.getInertiaData();

std::cout << g.angleDegrees() << std::endl;
std::cout << g.angularVelocityDegreesPerSecond() << std::endl;

std::cout << robot.getHeading() << std::endl;
std::cout << robot.getAngularVelocity() << std::endl;
```

main.cpp – Demo Application Walkthrough

Lists COM ports and asks the user to choose one.

Creates Parameters and Kobuki objects.

Calls init() and enable().

Waits for incoming sensor packets.

Prints sensors, gyro values, and odometry.

Runs a keyboard control loop until Q is pressed.

```
kobuki::Parameters parameters;  
parameters.device_port = port;  
parameters.command_rate_hz = 20;  
  
kobuki::Kobuki robot;  
robot.init(parameters);  
robot.enable();
```

Usage Example 1 – Basic Motion Commands

```
kobuki::Parameters p;  
p.device_port = "COM4";  
  
kobuki::Kobuki robot;  
robot.init(p);  
robot.enable();  
  
robot.setBaseControl(0.15, 0.0);    // move forward  
  
std::this_thread::sleep_for(std::chrono::seconds(2));  
robot.setBaseControl(0.0, 0.8);    // rotate left  
  
std::this_thread::sleep_for(std::chrono::seconds(1));  
robot.setBaseControl(0.0, 0.0);    // stop  
robot.disable();
```

Linear velocity is in m/s.

Angular velocity is in rad/s.

The command loop keeps sending the current target command at a fixed rate.

Usage Example 2 – Reading Sensors, Gyro, and Odometry

```
auto sensors = robot.getCoreSensorData();  
auto gyro    = robot.getInertiaData();  
auto odom    = robot.getOdometryData();  
  
std::cout << sensors.left_encoder << std::endl;  
std::cout << sensors.right_encoder << std::endl;  
std::cout << sensors.batteryVoltage() << std::endl;  
  
std::cout << gyro.angleRadians() << std::endl;  
std::cout << robot.getHeading() << std::endl;  
  
std::cout << odom.x << ", " << odom.y << std::endl;  
std::cout << odom.heading << std::endl;
```

Use getters to retrieve the latest values at any time.

The reader thread updates these values in the background.

This pattern is simple and effective for teaching.

Resetting and Interpreting Motion Data

```
robot.resetOdometry();
```

`resetOdometry()` sets `x`, `y`, `heading`, `left_distance`, and `right_distance` back to zero.

It also resets the gyro heading reference so that future `getHeading()` values are relative to the new zero.

This is useful before a classroom experiment or path-following demo.

Interpretation example

If `left_distance` and `right_distance` increase by similar amounts, the robot moved mostly straight. If one side increases more than the other, the robot turned.

Another example

If `left_distance` is negative and `right_distance` is positive, the robot is rotating in place.

Kobuki.h

The public robot class used by the application

```
struct Parameters {  
    std::string device_port = "COM3";  
    unsigned int command_rate_hz = 20;  
};  
  
class Kobuki {  
public:  
    void init(Parameters& parameters);  
    bool enable();  
    bool disable();  
  
    bool isEnabled() const;  
    bool isAlive() const;  
    bool isConnected() const;  
  
    void setBaseControl(const double& linearVelocity,  
                        const double& angularVelocity);  
    CoreSensors::Data getCoreSensorData() const;  
};
```

This is the main class students interact with.

It hides serial-port and packet details.

The API is intentionally small for teaching.

The destructor sends a stop command when the object is destroyed.

Kobuki.cpp

Implementation of the robot class

init()

Opens the COM port and starts background threads.

enable()

Allows the command loop to send movement commands.

disable()

Sends stop command and disables movement.

readerLoop()

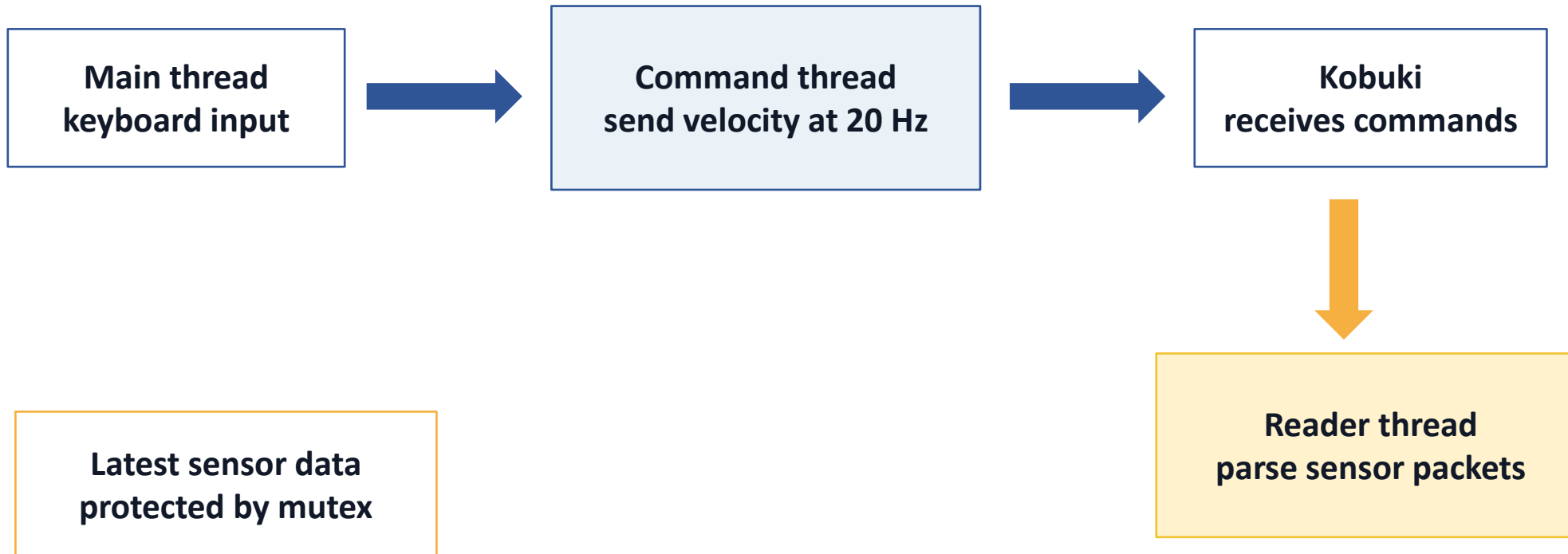
Reads serial bytes and updates the latest sensor data.

commandLoop()

Periodically sends the current target velocity.

Background Threads

Why the project uses a reader thread and a command thread



Keyboard input should not block sensor reading.
Commands are repeated at a stable rate.
Mutexes protect shared sensor and command data.

Keyboard Control Loop

Mapping keys to robot commands

```
while (true) {  
    const int key = _getch();  
    switch (key) {  
        case 'w': case 'W':  
            robot.setBaseControl(0.15, 0.0);  
            break;  
        case 'a': case 'A':  
            robot.setBaseControl(0.0, 0.8);  
            break;  
        case 'x': case 'X': case ' ':  
            robot.setBaseControl(0.0, 0.0);  
            break;  
        case 'q': case 'Q':  
            robot.disable();  
            return 0;  
    }  
}
```

Teaching points

`_getch()` reads one key without requiring Enter.

Each key changes the target velocity. Stopping is always available using X or Space.

Q stops the robot before exiting.

printSensors()

Displaying live robot feedback

```
void printSensors(const kobuki::CoreSensors::Data& d) {  
    std::cout << "Left encoder  : " << d.left_encoder << "\n";  
    std::cout << "Right encoder : " << d.right_encoder << "\n";  
    std::cout << "Battery      : " << d.batteryVoltage() << " V\n";  
    std::cout << "Bumper L/C/R  : "  
        << d.bumperLeft() << "/"  
        << d.bumperCenter() << "/"  
        << d.bumperRight() << "\n";  
}
```

Encoders help students understand wheel motion.

Battery voltage is useful before lab activities.

Bumper and cliff values are simple safety inputs.

The R key prints a snapshot of the latest data.

Extension idea: print sensors every second while the robot moves.

Usage Example 1: Move Forward and Stop

A minimal program fragment for the first lab

```
kobuki::Parameters parameters;  
parameters.device_port = "COM4";  
  
kobuki::Kobuki robot;  
robot.init(parameters);  
robot.enable();  
  
robot.setBaseControl(0.15, 0.0);  
std::this_thread::sleep_for(std::chrono::seconds(2));  
  
robot.setBaseControl(0.0, 0.0);  
robot.disable();
```

Expected behavior

The robot moves forward for two seconds.
Then it sends a stop command.
This is the safest first movement test.

Use a low speed such as 0.10–0.15 m/s for
the first classroom demo.

Usage Example 2: Stop on Bumper

Using sensor feedback in a simple safety behavior

```
robot.setBaseControl(0.10, 0.0);

while (true) {
    auto data = robot.getCoreSensorData();

    if (data.bumperLeft() ||
        data.bumperCenter() ||
        data.bumperRight()) {
        robot.setBaseControl(0.0, 0.0);
        std::cout << "Bumper pressed: stop!\n";
        break;
    }

    std::this_thread::sleep_for(
        std::chrono::milliseconds(50));
}
```

Concepts introduced

Polling sensor data

Boolean helper functions

Event-like robot behavior

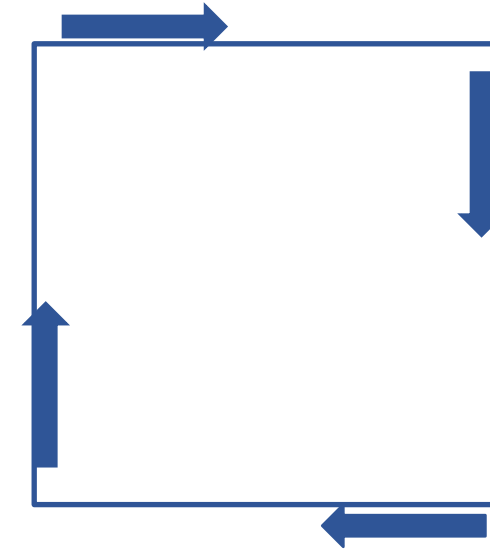
Emergency stop logic

This example connects C++ if-statements to physical robot behavior.

Usage Example 3: Simple Square Motion

A structured sequence of movements

```
for (int side = 0; side < 4; ++side) {  
    robot.setBaseControl(0.12, 0.0);  
    std::this_thread::sleep_for(std::chrono::seconds(2));  
  
    robot.setBaseControl(0.0, 0.0);  
    std::this_thread::sleep_for(std::chrono::milliseconds(400));  
  
    robot.setBaseControl(0.0, 0.8);  
    std::this_thread::sleep_for(std::chrono::milliseconds(1900));  
  
    robot.setBaseControl(0.0, 0.0);  
    std::this_thread::sleep_for(std::chrono::milliseconds(400));  
}
```



This is open-loop motion. It depends on floor friction, battery, and timing.

Homw Work

Move forward for exactly 2 m turn 360 degree then return to its initial position.

Move Forward until hit an obstacle then move backward for 1 m.

Mover around a square with 1 m borders stop when return to initial position.

Mover around a square with $r=2$ m, stop when return to initial position.